

Documentation_GitHub_Actions

GTACD - GitHub Actions CI/CD

Dans ce projet, nous avons construit un pipeline CI/CD complet avec GitHub Actions, en plaçant la sécurité au cœur de chaque étape. À chaque push sur `main`, un workflow automatique vérifie l'application, valide la configuration SSH, puis déclenche un déploiement via Ansible sur une VM Debian. Les secrets (clé SSH, IP serveur, credentials) sont gérés exclusivement via GitHub Secrets et ne transitent jamais en clair. Le serveur cible est durci avec un utilisateur `deployuser` à moindre privilège et une clé SSH dédiée au projet. L'ensemble forme une chaîne traçable et reproductible : chaque modification de code se retrouve en production de façon automatisée, auditée et sécurisée.

Sommaire

1. [Concepts clés](#)
 2. [Application et script de vérification](#)
 3. [Premier Workflow CI](#)
 4. [Gestion des Secrets](#)
 5. [Préparation du serveur de déploiement](#)
 6. [Playbook Ansible de déploiement](#)
 7. [Intégration Ansible dans GitHub Actions \(CD\)](#)
 8. [Test du cycle CI/CD complet](#)
 9. [Bilan et perspectives](#)
-

Concepts clés

CI/CD désigne trois pratiques complémentaires :

- **Intégration Continue (CI)** : chaque push déclenche automatiquement des tests pour détecter les régressions au plus tôt.
- **Livraison Continue** : le code validé est automatiquement packagé et prêt à être déployé.
- **Déploiement Continu (CD)** : le déploiement en production est lui aussi automatisé, sans intervention manuelle.

Pourquoi c'est indispensable en cybersécurité : les pipelines CI/CD sont devenus une cible privilégiée (attaques supply chain). Automatiser correctement réduit les erreurs

humaines et garantit la traçabilité de chaque déploiement, essentielle pour l'audit et la réponse aux incidents.

Concepts GitHub Actions :

Terme	Rôle
Workflow	Pipeline défini dans un fichier <code>.yaml</code>
Event	Déclencheur (push, pull_request...)
Job	Ensemble d'étapes exécutées sur un runner
Step	Commande ou action unitaire
Runner	Machine virtuelle qui exécute le job
Action	Brique réutilisable (ex : actions/checkout)
Secret	Variable chiffrée, masquée dans les logs

[Sommaire](#)

Application et script de vérification

Nous créons ici la base concrète du projet : un dépôt GitHub public, une application web minimaliste en HTML et un script Bash qui vérifie la présence du fichier `index.html` . Ce script simule un test automatisé et retourne un code de sortie `0` en cas de succès ou `1` en cas d'échec. Ces deux fichiers constituent la cible que le pipeline CI/CD va tester et déployer tout au long du projet.

Structure :

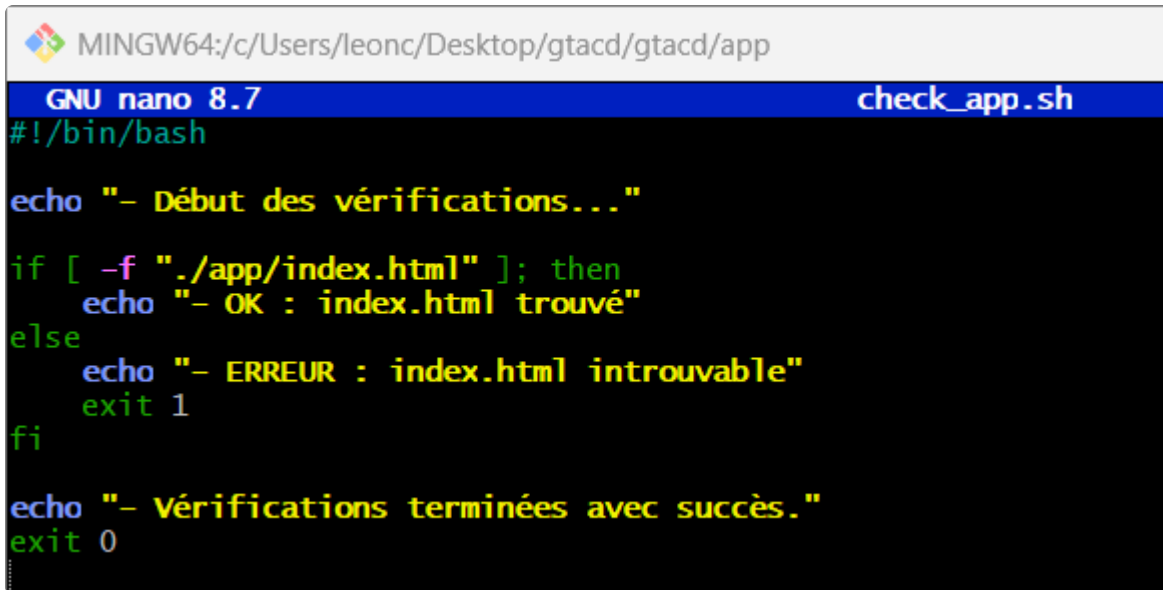
```
app/
├─ index.html
└─ check_app.sh
```

`app/index.html`

```
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="UTF-8"><title>TP CI/CD</title></head>
<body>
  <h1>Application TP GitHub Actions</h1>
  <p>Bienvenue sur notre application de démonstration CI/CD.</p>
  <!-- Date de génération : {{ generated_date }} -->
```

```
</body>  
</html>
```

app/check_app.sh



```
MINGW64:/c/Users/leonc/Desktop/gtacd/gtacd/app  
GNU nano 8.7 check_app.sh  
#!/bin/bash  
  
echo "- Début des vérifications..."  
  
if [ -f "./app/index.html" ]; then  
    echo "- OK : index.html trouvé"  
else  
    echo "- ERREUR : index.html introuvable"  
    exit 1  
fi  
  
echo "- Vérifications terminées avec succès."  
exit 0
```

Création du dépôt GitHub public et premier push :

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

1 General

Owner *



leon-camuzat



Repository name *

gtacd

✔ gtacd is available.

Great repository names are short and memorable. How about [cautious-spoon](#)?

Description

0 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository



Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off



Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository



gtacd

Public



0



0



0



Set up GitHub Copilot

Use GitHub's AI pair programmer to autocomplete suggestions as you code.

Get started with GitHub Copilot



Add collaborators to this repository

Search for people using their GitHub username or email address.

Invite collaborators

Quick setup — if you've done this kind of thing before

Set up in Desktop



HTTPS



SSH

<https://github.com/leon-camuzat/gtacd.git>



Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# gtacd" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/leon-camuzat/gtacd.git
git push -u origin main
```



...or push an existing repository from the command line

```
MINGW64:/c/Users/leonc/Desktop/gtacd/gtacd
leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd
$ git clone https://github.com/leon-camuzat/gtacd.git
Cloning into 'gtacd'...
warning: You appear to have cloned an empty repository.

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd
$ cd gtacd

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)
$ ls

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)
$ pwd
/c/Users/leonc/Desktop/gtacd/gtacd

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)
$ |
```

[Sommaire](#)

Premier Workflow CI

Nous mettons en place le premier workflow GitHub Actions dans `.github/workflows/ci_basic.yml`, qui se déclenche automatiquement à chaque push sur la branche `main`. Il contient un seul job `build-and-test` qui clone le dépôt, exécute le script de vérification et affiche le contenu de `index.html` dans les logs. C'est la première ligne de défense automatisée : aucune modification ne passe sans être vérifiée.

Fichier `.github/workflows/ci_basic.yml` :

```
name: Basic CI Workflow

on:
  push:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
```

```
- name: Run application check script
  run: |
    chmod +x ./app/check_app.sh
    ./app/check_app.sh

- name: Display index.html content
  run: cat app/index.html
```

Après le push, le workflow se déclenche automatiquement depuis l'onglet **Actions** :

```

MINGW64:/c:/Users/leonc/Desktop/gtacd/gtacd/.github/workflows
GNU nano 8.7 ci_basic.yml Modified
# Nom du workflow affiché dans l'onglet Actions de GitHub
name: Basic CI Workflow

# Définit quand le workflow se déclenche
on:
  push:          # Événement : un push est effectué
    branches:    # Uniquement sur la branche main
      - main

jobs:
  build-and-test: # Nom du job (libre)
    runs-on: ubuntu-latest # GitHub loue une VM Ubuntu vierge pour ce job

    steps:
      # Step 1 : clone le dépôt GitHub sur la VM du runner
      # Sans cette étape, les fichiers index.html et check_app.sh n'existent pas sur le runner
      - name: Checkout repository
        uses: actions/checkout@v4

      # Step 2 : exécute le script de vérification
      # Si le script retourne exit 1, le pipeline s'arrête ici
      - name: Run application check script
        run: bash ./app/check_app.sh

      # Step 3 : affiche le contenu de index.html dans les logs
      # Permet de vérifier visuellement ce qui est présent sur le runner
      - name: Display index.html content
        run: cat app/index.html

```

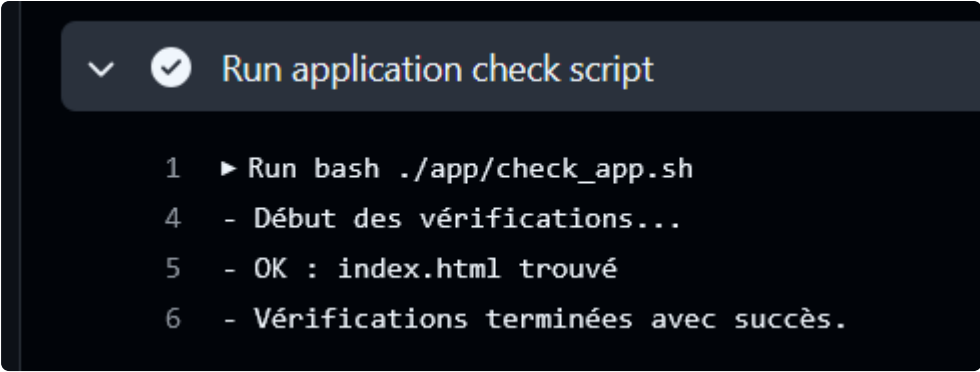
> Annotations

1 warning

build-and-test

succeeded 3 minutes ago in 4s

- >  Set up job
- >  Checkout repository
- >  Run application check script
- >  Display index.html content
- >  Post Checkout repository
- >  Complete job



```
1 ▶ Run bash ./app/check_app.sh
4 - Début des vérifications...
5 - OK : index.html trouvé
6 - Vérifications terminées avec succès.
```

[Sommaire](#)

Gestion des Secrets

Nous abordons ici la gestion sécurisée des informations sensibles via GitHub Secrets, en créant deux secrets fictifs (`FAKE_API_TOKEN` et `ENV_TYPE`) et en les injectant dans le workflow. Nous observons comment GitHub masque automatiquement leurs valeurs dans les logs. Nous simulons également un échec volontaire du pipeline pour apprendre à lire et diagnostiquer les logs d'erreur.

Pourquoi utiliser les Secrets GitHub ?

Les secrets ne doivent **jamais** être écrits en clair dans le code ou les fichiers de config : un dépôt public expose instantanément la valeur à n'importe qui. Un secret commité par erreur reste dans l'historique git même après suppression. GitHub Secrets chiffre les valeurs et les masque automatiquement dans tous les logs.

Risques d'exposition :

- Prise de contrôle de services tiers (API, cloud)
- Accès non autorisé à l'infrastructure
- Pivoting dans le SI via les credentials volés

Configuration des secrets

Dans **Settings** → **Secrets and variables** → **Actions**, deux secrets sont créés :

- `FAKE_API_TOKEN` : valeur fictive simulant un token d'API
- `ENV_TYPE` : valeur `DEV`

Actions secrets and variables

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

Secrets

Variables

Environment secrets

This environment has no secrets.

[Manage environment secrets](#)

Repository secrets

This repository has no secrets.

[New repository secret](#)

Actions secrets / New secret

Name *

FAKE_API_TOKEN

Secret *

tok-abc123fake456

[Add secret](#)

Repository secrets

[New repository secret](#)

Name 	Last updated	
 ENV_TYPE	now	 
 FAKE_API_TOKEN	now	 

Intégration dans le workflow

Ajout d'une étape dans `ci_basic.yml` :

```
- name: Display environment info
  env:
    API_TOKEN: ${ secrets.FAKE_API_TOKEN }
    ENV: ${ secrets.ENV_TYPE }
```

```
APP_NAME: "mon-app-ci"
run: |
  echo "Environnement : $ENV"
  echo "Token API : $API_TOKEN"
  echo "Nom app : $APP_NAME"
  echo "Runner OS : $RUNNER_OS"
```

Observation : GitHub censure automatiquement la valeur des secrets dans les logs.

```

MINGW64:/c/Users/leonc/Desktop/gtacd/gtacd/.github/workflows
GNU nano 8.7 ci_basic.yml
# Nom du workflow affiché dans l'onglet Actions de GitHub
name: Basic CI Workflow

# Définit quand le workflow se déclenche
on:
  push:          # Événement : un push est effectué
    branches:
      - main      # Uniquement sur la branche main

jobs:
  build-and-test: # Nom du job (libre)
    runs-on: ubuntu-latest # GitHub loue une VM Ubuntu vierge pour ce job

    steps:
      # Step 1 : clone le dépôt GitHub sur la VM du runner
      # Sans cette étape, les fichiers index.html et check_app.sh n'existent pas sur le runner
      - name: Checkout repository
        uses: actions/checkout@v4

      # Step 2 : exécute le script de vérification
      # Si le script retourne exit 1, le pipeline s'arrête ici
      - name: Run application check script
        run: bash ./app/check_app.sh

      # Step 3 : affiche le contenu de index.html dans les logs
      # Permet de vérifier visuellement ce qui est présent sur le runner
      - name: Display index.html content
        run: cat app/index.html

# Step 4 : affiche les variables d'environnement et les secrets dans les logs
# Les secrets apparaissent censurés avec *** dans les logs GitHub
- name: Display environment variables and secrets
  env:
    FAKE_API_TOKEN: ${ secrets.FAKE_API_TOKEN } # secret GitHub
    ENV_TYPE: ${ secrets.ENV_TYPE } # secret GitHub
    APP_NAME: gtacd # variable personnalisée non-secrète
  run: |
    echo "Token API : $FAKE_API_TOKEN"
    echo "Type d'environnement : $ENV_TYPE"
    echo "Nom de l'application : $APP_NAME"
    echo "OS du runner : $RUNNER_OS"

```

All workflows Filter workflow runs

Showing runs from all workflows

Help us improve GitHub Actions
 Tell us how to make GitHub Actions work better for you with three quick questions.
 Give feedback

2 workflow runs

	Workflow	Event	Status	Branch	Actor
ci: ajout de l'affichage des secrets et variables d'environnement Basic CI Workflow #2: Commit 8df259a pushed by leon-camuzat	main		1 minute ago 9s	...	
ci: add basic CI workflow Basic CI Workflow #1: Commit e50e89b pushed by leon-camuzat	main		44 minutes ago 10s	...	

← Basic CI Workflow

✅ ci: ajout de l'affichage des secrets et variables d'environnement #2 Re-run all jobs ...

Summary

All jobs

build-and-test

Run details

Usage

Workflow file

Triggered via push 1 minute ago

leon-camuzat pushed [8d259a](#) [main](#)

Status: **Success**

Total duration: **9s**

Artifacts: —

ci_basic.yml

on: push

build-and-test 4s

Annotations

1 warning

⚠ build-and-test
Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4. For more information see: —
[Show more](#)

build-and-test

succeeded 1 minute ago in 4s

- > ✓ Set up job
- > ✓ Checkout repository
- > ✓ Run application check script
- > ✓ Display index.html content
- ✓ ✓ **Display environment variables and secrets**
 - 1 ▶ Run echo "Token API : \$FAKE_API_TOKEN"
 - 11 Token API : ***
 - 12 Type d'environnement : ***
 - 13 Nom de l'application : gtacd
 - 14 OS du runner : Linux
- > ✓ Post Checkout repository
- > ✓ Complete job

Simulation d'un échec contrôlé

Modification de `check_app.sh` pour forcer `exit 1`, puis retour à `exit 0` après observation :

MINGW64:/c/Users/leonc/Desktop/gtacd/gtacd/app

GNU nano 8.7

check_app.sh

#!/bin/bash

echo "- Début des vérifications..."

if [-f "./app/index.html"]; then

echo "- OK : index.html trouvé"

else

echo "- ERREUR : index.html introuvable"

exit 1

fi

echo "- Vérifications terminées avec succès."

exit 1

MINGW64:/c/Users/leonc/Desktop/gtacd/gtacd

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd/app (main)

\$ cd ..

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)

\$ ls

app/

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)

\$ git add .

warning: in the working copy of 'app/check_app.sh', LF will be replaced by CRLF

warning: in the working copy of 'app/index.html', LF will be replaced by CRLF

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)

\$ git commit -m "test: simulation d'un echec volontaire du pipeline"

[main 9aca14c] test: simulation d'un echec volontaire du pipeline

1 file changed, 1 insertion(+), 1 deletion(-)

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)

\$ git push origin main

Enumerating objects: 7, done.

Counting objects: 100% (7/7), done.

Delta compression using up to 18 threads

Compressing objects: 100% (4/4), done.

Writing objects: 100% (4/4), 411 bytes | 411.00 KiB/s, done.

Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)

remote: Resolving deltas: 100% (1/1), completed with 1 local object.

To https://github.com/leon-camuzat/gtacd.git

8df259a..9aca14c main -> main

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)

\$ |

All workflows Filter workflow runs

Showing runs from all workflows

Help us improve GitHub Actions
Tell us how to make GitHub Actions work better for you with three quick questions. Give feedback ×

3 workflow runs Workflow ▾ Event ▾ Status ▾ Branch ▾ Actor ▾

✗ test: simulation d'un echec volontaire du pipeline Basic CI Workflow #3: Commit 9aca14c pushed by leon-camuzat	main	📅 now 🕒 14s	⋮
✓ ci: ajout de l'affichage des secrets et variables d'environnement Basic CI Workflow #2: Commit 8df259a pushed by leon-camuzat	main	📅 9 minutes ago 🕒 9s	⋮
✓ ci: add basic CI workflow Basic CI Workflow #1: Commit e50e89b pushed by leon-camuzat	main	📅 53 minutes ago 🕒 10s	⋮

✓ ✗ **Run application check script**

```

1 ▶ Run bash ./app/check_app.sh
4 - Début des vérifications...
5 - OK : index.html trouvé
6 - Vérifications terminées avec succès.
7 Error: Process completed with exit code 1.

```

```

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)
$ git commit -m "fix: remise en etat du script de verification"
[main b8e426d] fix: remise en etat du script de verification
1 file changed, 2 insertions(+), 1 deletion(-)

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)
$ git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 18 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 408 bytes | 408.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/leon-camuzat/gtacd.git
9aca14c..b8e426d main -> main

```

[Sommaire](#)

Préparation du serveur de déploiement

Nous configurons la VM Debian cible en créant un utilisateur `deployuser` à moindre privilège avec des droits sudo sans mot de passe. Nous générons une paire de clés SSH dédiée au projet, sans passphrase, et l'installons sur la VM. Un workflow de simulation

valide l'ensemble de la configuration SSH et des secrets sans nécessiter de connexion réseau réelle.

Pourquoi un utilisateur dédié avec moindre privilège ?

Utiliser `root` directement pour le déploiement est une faute critique : toute compromission du pipeline donne un accès total au serveur. Un utilisateur `deployuser` limité contient la surface d'attaque, si la clé SSH est volée, l'attaquant ne peut faire que ce que `deployuser` est autorisé à faire.

Configuration de la VM Debian

```
root@test:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 1000
    link/ether 00:0c:29:1c:cc:65 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c291ccc65
    inet 192.168.241.129/24 brd 192.168.241.255 scope global dynamic noprefixroute ens33
        valid_lft 1793sec preferred_lft 1568sec
    inet6 fe80::928f:cb4c:4e1e:8d9c/64 scope link
        valid_lft forever preferred_lft forever
root@test:~# _
```

```
• ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-08-04 14:54:12 CEST; 1min 39s ago
 Invocation: f9fe8d8781ff41e1abfbd200fa4b8509
    Docs: man:sshd(8)
          man:sshd_config(5)
   Process: 720 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
  Main PID: 763 (sshd)
    Tasks: 1 (limit: 1093)
   Memory: 2.8M (peak: 3.3M)
      CPU: 112ms
   CGroup: /system.slice/ssh.service
           └─763 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

août 04 14:54:11 test systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
août 04 14:54:12 test sshd[763]: Server listening on 0.0.0.0 port 22.
août 04 14:54:12 test sshd[763]: Server listening on :: port 22.
août 04 14:54:12 test systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
root@test:~#
```

```
root@test:~# adduser deployuser
Nouveau mot de passe :
Retapez le nouveau mot de passe :
passwd : mot de passe mis à jour avec succès
Modifier les informations associées à un utilisateur pour deployuser
Entrer la nouvelle valeur, ou appuyer sur ENTER pour la valeur par défaut
  NOM []: deployuser
  Numéro de chambre []:
  Téléphone professionnel []:
  Téléphone personnel []:
  Autre []:
Is the information correct? [Y/n] y
root@test:~# _
```

```

root@test:~# apt install sudo
Installation de :
  sudo

Sommaire :
  Mise à niveau de : 0. Installation de : 1 Supprimé : 0. Non mis à jour : 0
  Taille du téléchargement : 2 086 kB
  Espace nécessaire : 6 866 kB / 8 043 MB disponible

Réception de : 1 http://deb.debian.org/debian trixie/main amd64 sudo amd64 1.9.16p2-3+deb13u2 [2 086 kB]
2 086 ko réceptionnés en 3s (647 ko/s)
Sélection du paquet sudo précédemment désélectionné.
(Lecture de la base de données... 36058 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../sudo_1.9.16p2-3+deb13u2_amd64.deb ...
Dépaquetage de sudo (1.9.16p2-3+deb13u2) ...
Paramétrage de sudo (1.9.16p2-3+deb13u2) ...
Traitement des actions différées (« triggers ») pour man-db (2.13.1-1) ...
Traitement des actions différées (« triggers ») pour libc-bin (2.41-12+deb13u3) ...
root@test:~#

```

```

# See sudoers(5) for more information on "@include" directives:

@includedir /etc/sudoers.d

deployuser ALL=(ALL) NOPASSWD:ALL

```

```

root@test:~# su - deployuser
deployuser@test:~$ whoami
deployuser
deployuser@test:~$ sudo whoami
root
deployuser@test:~$

```

Génération de la clé SSH dédiée

Pourquoi une clé dédiée par projet ? Si une clé est compromise, on la révoque sans impacter les autres projets ni les accès personnels. La rotation est isolée et traçable.

```

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop
$ ssh-keygen -t rsa -b 4096 -f ~/.ssh/github_deploy_key -N ""
Generating public/private rsa key pair.
Your identification has been saved in /c/Users/leonc/.ssh/github_deploy_key
Your public key has been saved in /c/Users/leonc/.ssh/github_deploy_key.pub
The key fingerprint is:
SHA256:0ZTl4ReqMCqrPQuuKXAcQFUoZd5H0lJdAq1xhKu9pak leonc@C56EK3PBREMR5
The key's randomart image is:
+---[RSA 4096]-----+
|..o+ooo+*oo+o . |
|..o...o+ *oo o . |
|... ..o . + . |
|. . .+ + . . |
|. . .+ S . |
|. o + . . |
|... . = |
|.o +. + |
|+.o oE. |
+-----[SHA256]-----+

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop
$

```

```

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop
$ cat ~/.ssh/github_deploy_key.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCP+A1+dcsVrgeH3LVJblcNlxFk/aoiSZySh0gLEH9nUsgkMgCgJqm7GCUlA+20hdB
HfpjVIK04QeXbV2TM9RzR0zyUljjABC7IFJAHQDi2/H52bUC1rJFa5xmoN06CN0xmNvfLt4aCCSCdzABH2od591QszIH9fPIYrhPYlJ
lPzFfQF1mnA0HZzmxc3JN57IOhUc59SsCW0ZkiSRIhudw0SyY4RZCFdGg8D6GExVj02/KlCQiBMiShAwwzS0nn73lrtl2U7/JczplI2
oHGuHDNBm4pAv6o0YHUc5LtYuiKGcJn0Sdw8hKQd206YTKu+cLIOMeOUHi6R9PujKdSjHTfAA6bmWHSZ4f0FUh0zBshMUdkHcXmXvnF
hWuK6mdJr3Fk6w41Nxdwk0RQRuZ+l66BTjkFYna4x1HawP1dB1GvPeAjwnrL3LkFVAY9+JexWboNb3Lj1v290dzMnrbdhZXWsdT3zia
WrZ06GZReia2+QzV/CVap6uH4Gm/EMdQIPEAZylzwywI/+cbqoQJcnx/SHFQ4okEJ26fKFdpa7RbgRCAXBygmyDNPe4M4p/lx2c2+Fb
AaRXfouNd7dk6fQKeF1Nr97pdKeHQSJYPuuGakiQUNf28BnQDc50QCJvu14T+GqHZLiQ5GfanVylz72SgAQHC/cuodIByZNVB41HCKz
w== leonc@C56EK3PBREMR5

```

```

deployuser@test:~$ mkdir -p ~/.ssh
chmod 700 ~/.ssh
echo "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCP+A1+dcsVrgeH3LVJblcNlxFk/aoiSZySh0gLEH9nUsgkMgCgJqm7GCUlA+20hdB
HfpjVIK04QeXbV2TM9RzR0zyUljjABC7IFJAHQDi2/H52bUC1rJFa5xmoN06CN0xmNvfLt4aCCSCdzABH2od591QszIH9fPIYrhPYlJlPzFfQF1mnA0HZzm
xc3JN57IOhUc59SsCW0ZkiSRIhudw0SyY4RZCFdGg8D6GExVj02/KlCQiBMiShAwwzS0nn73lrtl2U7/JczplI2oHGuHDNBm4pAv6o0YHUc5LtYuiKGcJn0Sdw8hKQd206YTKu+cLIOMeOUHi6R9PujK
dSjHTfAA6bmWHSZ4f0FUh0zBshMUdkHcXmXvnFhWuK6mdJr3Fk6w41Nxdwk0RQRuZ+l66BTjkFYna4x1HawP1dB1GvPeAjwnrL3LkFVAY9+JexWboNb3Lj1v290dzMnrbdhZXWsdT3ziaWrZ06GZReia
2+QzV/CVap6uH4Gm/EMdQIPEAZylzwywI/+cbqoQJcnx/SHFQ4okEJ26fKFdpa7RbgRCAXBygmyDNPe4M4p/lx2c2+FbAaRXfouNd7dk6fQKeF1Nr97pdKeHQSJYPuuGakiQUNf28BnQDc50QCJvu14T
+GqHZLiQ5GfanVylz72SgAQHC/cuodIByZNVB41HCKzw== leonc@C56EK3PBREMR5" >> ~/.ss
h/authorized_keys
chmod 600 ~/.ssh/authorized_keys
deployuser@test:~$ |

```

Windows PowerShell

Copyright (C) Microsoft Corporation. All rights reserved.

```

PS C:\Users\leonc> ssh -i ~/.ssh/github_deploy_key deployuser@192.168.241.12
9
Linux test 6.12.100+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.100-1 (20
26-07-30) x86_64

```

The programs included with the Debian GNU/Linux system are free software; the exact distribution terms for each program are described in the individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent permitted by applicable law.

```

deployuser@test:~$ |

```

Secrets GitHub pour le déploiement

Dans **Settings** → **Secrets and variables** → **Actions**, ajout de :

- `SSH_PRIVATE_KEY` : contenu complet de `~/.ssh/github_deploy_key`
- `SERVER_IP` : adresse IP de la VM
- `SERVER_USER` : `deployuser`

Repository secrets			New repository secret	
Name 			Last updated	
 <code>ENV_TYPE</code>			3 hours ago	 
 <code>FAKE_API_TOKEN</code>			3 hours ago	 
 <code>SERVER_IP</code>			now	 
 <code>SERVER_USER</code>			now	 
 <code>SSH_PRIVATE_KEY</code>			1 minute ago	 

Workflow de simulation

Fichier `.github/workflows/secure-deployment-simulation.yml` — valide la configuration sans connexion réseau réelle :

```

name: Secure Deployment Simulation

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  validate-deployment-config:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Validate SSH private key format
        env:
          SSH_KEY: ${ secrets.SSH_PRIVATE_KEY }
        run: |
          echo "$SSH_KEY" > /tmp/test_key

```

```

chmod 600 /tmp/test_key
# Vérification du format PEM
if head -1 /tmp/test_key | grep -q "BEGIN"; then
    echo "[OK] Clé SSH au bon format"
else
    echo "[ERREUR] Format de clé invalide"; exit 1
fi
# Vérification taille (>= 2048 bits)
ssh-keygen -l -f /tmp/test_key
# Vérification absence de passphrase
ssh-keygen -y -f /tmp/test_key > /dev/null && echo "[OK] Pas de
passphrase"
rm /tmp/test_key

- name: Validate server secrets
  env:
    SERVER_IP: ${ secrets.SERVER_IP }
    SERVER_USER: ${ secrets.SERVER_USER }
  run: |
    [ -n "$SERVER_IP" ] && echo "[OK] SERVER_IP défini" || (echo "[ERREUR] SERVER_IP manquant"; exit 1)
    [ -n "$SERVER_USER" ] && echo "[OK] SERVER_USER défini" || (echo "[ERREUR] SERVER_USER manquant"; exit 1)

- name: Simulate SSH deployment commands
  env:
    SERVER_IP: ${ secrets.SERVER_IP }
    SERVER_USER: ${ secrets.SERVER_USER }
  run: |
    echo "=== Simulation des commandes de déploiement ==="
    echo "Cible : $SERVER_USER@$SERVER_IP"
    echo "Commandes qui seraient exécutées :"
    echo "  ansible-playbook -i inventory.ini ansible/deploy.yml"
    echo "  nginx -t && systemctl reload nginx"
    echo "[SIMULATION] Déploiement validé"

- name: Security checks
  run: |
    echo "=== Contrôles de sécurité ==="
    echo "[OK] Secrets utilisés via variables d'environnement"
    echo "[OK] Aucun secret en clair dans le code"
    echo "[OK] Utilisateur non-root pour le déploiement"
    echo "[OK] Clé SSH dédiée au projet"

```

```

MINGW64:/c:/Users/leonc/Desktop/gtacd/gtacd
GNU nano 8.7 .github/workflows/secure-deployment-simulation.yml Modified
name: Secure Deployment Simulation

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  simulate-deployment:
    runs-on: ubuntu-latest

    steps:
      # Checkout du dépôt
      - name: Checkout repository
        uses: actions/checkout@v4

      # Validation de la clé SSH privée
      - name: Validate SSH private key
        env:
          SSH_PRIVATE_KEY: ${ secrets.SSH_PRIVATE_KEY }
        run: |
          echo "$SSH_PRIVATE_KEY" > /tmp/test_key
          chmod 600 /tmp/test_key
          ssh-keygen -l -f /tmp/test_key
          echo "- Format de la cle : OK"
          ssh-keygen -y -f /tmp/test_key > /dev/null 2>&1 && echo "- Absence de passphrase : OK"
          rm /tmp/test_key

      # Validation de la configuration serveur
      - name: Validate server configuration
        env:
          SERVER_IP: ${ secrets.SERVER_IP }
          SERVER_USER: ${ secrets.SERVER_USER }
        run: |
          echo "- SERVER_IP defini : $SERVER_IP"
          echo "- SERVER_USER defini : $SERVER_USER"
          echo "- Validation du format IP..."
          echo "$SERVER_IP" | grep -E '^([0-9]+\.[0-9]+\.[0-9]+\.[0-9]+)$' && echo "- Format IP : OK"

      # Simulation de connexion SSH
      - name: Simulate SSH connection
        env:
          SERVER_IP: ${ secrets.SERVER_IP }
          SERVER_USER: ${ secrets.SERVER_USER }
        run: |
          echo "- Commandes qui seraient executees sur le serveur :"
          echo "  ssh $SERVER_USER@$SERVER_IP 'mkdir -p /var/www/html/tp-app'"
          echo "  ssh $SERVER_USER@$SERVER_IP 'sudo systemctl restart nginx'"
          echo "  scp -r ./app/* $SERVER_USER@$SERVER_IP:/var/www/html/tp-app/"
          echo "- Simulation terminee : OK"

      # Controles de securite
      - name: Security checks
        run: |
          echo "- Verification des bonnes pratiques :"
          echo "  - Cle SSH dediee au projet : OK"
          echo "  - Utilisateur non-root (deployuser) : OK"

AG Help      AO Write Out  AF Where Is  AK Cut       AT Execute   AC Location  M-U Undo
AX Exit      AR Read File  \ Replace   AU Paste     AJ Justify   A/ Go To Line M-E Redo

```

simulate-deployment

succeeded now in 6s

>  Checkout repositoryv  Validate SSH private key

```

1 ▶ Run echo "$SSH_PRIVATE_KEY" > /tmp/test_key
11 4096 SHA256:0ZT14ReqMCqrPQuuKXAcQFUoZd5H01JdAq1xhKu9pak leonc@C56EK3PBREMR5 (RSA)
12 - Format de la cle : OK
13 - Absence de passphrase : OK

```

v  Validate server configuration

```

1 ▶ Run echo "- SERVER_IP defini : $SERVER_IP"
11 - SERVER_IP defini : ***
12 - SERVER_USER defini : ***
13
14 - Validation du format IP...
15 ***
16 - Format IP : OK

```

v  Simulate SSH connection

```

1 ▶ Run echo "- Commandes qui seraient executees sur le serveur : "
12 - Commandes qui seraient executees sur le serveur :
13   ssh ***
14 @*** 'mkdir -p /var/www/html/tp-app'
15   ssh ***
16 @*** 'sudo systemctl restart nginx'
17   scp -r ./app/* ***
18 @***:/var/www/html/tp-app/
19 - Simulation terminee : OK

```

>  Security checks>  Post Checkout repository**Script de test local**

MINGW64:/c/Users/leonc/Desktop/gtacd/gtacd

```

GNU nano 8.7 test_local.sh
#!/bin/bash

SERVER_IP="192.168.241.129"
SERVER_USER="deployuser"
KEY_PATH="$HOME/.ssh/github_deploy_key"

echo "- Début des tests de validation locale..."
echo ""

# Test 1 : connexion SSH
echo "- Test 1 : connexion SSH avec la clé dédiée..."
ssh -i "$KEY_PATH" -o ConnectTimeout=5 -o BatchMode=yes "$SERVER_USER@$SERVER_IP" "echo '- Connexion SSH : OK'"

# Test 2 : exécution sudo
echo "- Test 2 : exécution de commandes sudo..."
ssh -i "$KEY_PATH" "$SERVER_USER@$SERVER_IP" "sudo whoami" | grep root && echo "- Sudo sans mot de passe : OK"

# Test 3 : création de répertoire
echo "- Test 3 : création de répertoire d'application..."
ssh -i "$KEY_PATH" "$SERVER_USER@$SERVER_IP" "mkdir -p /tmp/tp-app-test && echo '- Création répertoire : OK'"

# Test 4 : vérification des services
echo "- Test 4 : vérification des services système..."
ssh -i "$KEY_PATH" "$SERVER_USER@$SERVER_IP" "systemctl is-active ssh && echo '- Service SSH actif : OK'"

echo ""
echo "- Tous les tests sont terminés."

```

MINGW64:/c/Users/leonc/Desktop/gtacd/gtacd

```

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)
$ nano test_local.sh

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)
$ bash test_local.sh
- Début des tests de validation locale...

- Test 1 : connexion SSH avec la clé dédiée...
- Connexion SSH : OK
- Test 2 : exécution de commandes sudo...
root
- Sudo sans mot de passe : OK
- Test 3 : création de répertoire d'application...
- Création répertoire : OK
- Test 4 : vérification des services système...
active
- Service SSH actif : OK

- Tous les tests sont terminés.

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)
$

```

[Sommaire](#)

Playbook Ansible de déploiement

Nous écrivons le playbook Ansible `deploy.yml` qui installe Nginx, crée le répertoire de l'application, copie les fichiers et configure un virtual host via un template Jinja2. L'inventaire

local pointe vers la VM avec l'utilisateur `deployuser` et la clé SSH dédiée. Le playbook est testé et validé localement avant d'être intégré au pipeline.

Un playbook Ansible c'est un fichier qui liste des tâches à exécuter sur un serveur distant, dans l'ordre. Au lieu de se connecter à la main sur la VM et de taper les commandes une par une (installer Nginx, copier les fichiers, redémarrer le service...), on écrit tout ça une fois dans un fichier `.yaml` et Ansible s'en charge automatiquement via SSH. C'est l'équivalent d'un script, mais structuré et idempotent — c'est-à-dire qu'on peut le relancer plusieurs fois sans risque, il ne fait que ce qui n'est pas déjà en place.

ansible/deploy.yml

```
---
- name: Deploy application
  hosts: webserver
  become: yes

  tasks:
    - name: Install Nginx
      apt:
        name: nginx
        state: present
        update_cache: yes

    - name: Create application directory
      file:
        path: /var/www/html/tp-app
        state: directory
        owner: www-data
        group: www-data
        mode: '0755'

    - name: Copy application files
      copy:
        src: ../app/
        dest: /var/www/html/tp-app/
        owner: www-data
        group: www-data

    - name: Configure Nginx vhost
      template:
        src: nginx_vhost.conf.j2
        dest: /etc/nginx/sites-available/tp-app
      notify: Reload Nginx
```

```

- name: Enable vhost
  file:
    src: /etc/nginx/sites-available/tp-app
    dest: /etc/nginx/sites-enabled/tp-app
    state: link

handlers:
- name: Reload Nginx
  service:
    name: nginx
    state: reloaded

```

ansible/nginx_vhost.conf.j2

```

server {
    listen 80;
    server_name {{ ansible_host }};

    root /var/www/html/tp-app;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}

```

inventory.ini (local, à ajouter dans .gitignore)

```

[webservers]
<IP_VM> ansible_user=deployuser
ansible_ssh_private_key_file=~/.ssh/github_deploy_key

```

Exécution locale pour validation avant push :

MINGW64:/c/Users/leonc/Desktop/gtacd/gtacd/ansible

GNU nano 8.7

deploy.yml

```
---
- name: Déploiement de l'application GTACD
  hosts: webserver
  become: yes

  vars:
    app_dest: /var/www/html/tp-app
    server_ip: "{{ ansible_host }}"

  tasks:

    # Tâche 1 : installer Nginx
    - name: Installer Nginx
      apt:
        name: nginx
        state: present
        update_cache: yes

    # Tâche 2 : créer le répertoire de destination
    - name: Créer le répertoire de destination
      file:
        path: "{{ app_dest }}"
        state: directory
        owner: www-data
        group: www-data
        mode: '0755'

    # Tâche 3 : copier les fichiers de l'application
    - name: Copier les fichiers de l'application
      copy:
        src: ../app/
        dest: "{{ app_dest }}"
        owner: www-data
        group: www-data
        mode: '0644'

    # Tâche 4 : configurer le vhost Nginx via le template Jinja2
    - name: Configurer le vhost Nginx
      template:
        src: nginx_vhost.conf.j2
        dest: /etc/nginx/sites-available/tp-app
        mode: '0644'

    # Tâche 5 : activer le site
    - name: Activer le site Nginx
      file:
        src: /etc/nginx/sites-available/tp-app
        dest: /etc/nginx/sites-enabled/tp-app
        state: link

    # Tâche 6 : redémarrer Nginx
    - name: Redémarrer Nginx
      service:
        name: nginx
        state: restarted
```

```

MINGW64:/c/Users/leonc/Desktop/gtacd/gtacd/ansible
GNU nano 8.7 nginx_vhost.conf.j2
server {
    listen 80;
    server_name {{ server_ip }};

    root /var/www/html/tp-app;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    access_log /var/log/nginx/tp-app-access.log;
    error_log /var/log/nginx/tp-app-error.log;
}

```

```

MINGW64:/c/Users/leonc/Desktop/gtacd/gtacd/ansible
GNU nano 8.7 inventory.ini
[webservers]
192.168.241.129 ansible_user=deployuser ansible_ssh_private_key_file=~/.ssh/github_deploy_key

root@test:~# ansible --version
ansible [core 2.19.4]
  config file = None
  configured module search path = ['/root/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /root/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.13.5 (main, Jul 15 2026, 20:25:40) [GCC 14.2.0] (/usr/bin/python3)
  jinja version = 3.1.6
  pyyaml version = 6.0.2 (with libyaml v0.2.5)
root@test:~#

```

```

root@test:/tmp/gtacd# ansible-playbook -i ansible/inventory.ini ansible/deploy.yml

PLAY [Déploiement de l'application GTACD] *****

TASK [Gathering Facts] *****
[WARNING]: Host '192.168.241.129' is using the discovered Python interpreter at '/usr/bin/python3.13', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.19/reference_appendices/interpreter_discovery.html for more information.
ok: [192.168.241.129]

TASK [Installer Nginx] *****
changed: [192.168.241.129]

TASK [Cr  er le r  pertoire de destination] *****
changed: [192.168.241.129]

TASK [Copier les fichiers de l'application] *****
changed: [192.168.241.129]

TASK [Configurer le vhost Nginx] *****
changed: [192.168.241.129]

TASK [Activer le site Nginx] *****
changed: [192.168.241.129]

TASK [Red  marrer Nginx] *****
changed: [192.168.241.129]

PLAY RECAP *****
192.168.241.129 : ok=7  changed=6  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

root@test:/tmp/gtacd#

```



[Sommaire](#)

Int  gration Ansible dans GitHub Actions (CD)

Le workflow `ci_basic.yml` est renomm   `ci_cd_full.yml` et enrichi d'un second job `deploy`, conditionn   au succ  s du CI via `needs: build-and-test`. Ce job charge la cl   SSH depuis les secrets, g  n  re dynamiquement l'inventaire Ansible, installe Ansible sur le runner et ex  cute le playbook. Le d  ploiement est ainsi enti  rement automatis   et trac      chaque push.

Le fichier `ci_basic.yml` est renomm   `ci_cd_full.yml` et enrichi d'un job `deploy` conditionn   au succ  s du CI.

Jusqu'ici le playbook Ansible on le lan  ait    la main depuis sa machine. L   on le branche directement dans le pipeline GitHub Actions : quand un push est fait sur `main`, GitHub ex  cute d'abord les tests, et si tout passe, il lance automatiquement Ansible pour d  ployer sur la VM. Plus besoin d'intervenir manuellement, le code test   arrive tout seul sur le serveur.

.github/workflows/ci_cd_full.yml (parties ajoutées) :

```
deploy:
  runs-on: ubuntu-latest
  needs: build-and-test

  steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Set up SSH agent
      uses: webfactory/ssh-agent@v0.9.0
      with:
        ssh-private-key: ${ secrets.SSH_PRIVATE_KEY }

    - name: Create Ansible inventory
      env:
        SERVER_IP: ${ secrets.SERVER_IP }
        SERVER_USER: ${ secrets.SERVER_USER }
      run: |
        echo "[webservers]" > inventory.ini
        echo "$SERVER_IP ansible_user=$SERVER_USER" >> inventory.ini
        chmod 600 inventory.ini

    - name: Install Ansible
      run: pip install ansible

    - name: Run Ansible playbook
      env:
        SERVER_IP: ${ secrets.SERVER_IP }
        SERVER_USER: ${ secrets.SERVER_USER }
      run: |
        ansible-playbook -i inventory.ini ansible/deploy.yml \
          --extra-vars "server_ip=$SERVER_IP"
```

MINGW64:/c/Users/leonc/Desktop/gtacd/gtacd

```
leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)
$ cd /c/Users/leonc/Desktop/gtacd/gtacd
mv .github/workflows/ci_basic.yml .github/workflows/ci_cd_full.yml
nano .github/workflows/ci_cd_full.yml

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)
$
```

MINGW64:/c/Users/leonc/Desktop/gtacd/gtacd

```
GNU nano 8.7 .github/workflows/ci_cd_full.yml
# Nom du workflow affiché dans l'onglet Actions de GitHub
name: Basic CI Workflow

# Définit quand le workflow se déclenche
on:
  push:          # Événement : un push est effectué
    branches:    # Uniquement sur la branche main
      - main

jobs:
  build-and-test: # Nom du job (libre)
    runs-on: ubuntu-latest # GitHub loue une VM Ubuntu vierge pour ce job

    steps:
      # Step 1 : clone le dépôt GitHub sur la VM du runner
      # Sans cette étape, les fichiers index.html et check_app.sh n'existent pas sur le runner
      - name: Checkout repository
        uses: actions/checkout@v4

      # Step 2 : exécute le script de vérification
      # Si le script retourne exit 1, le pipeline s'arrête ici
      - name: Run application check script
        run: bash ./app/check_app.sh

      # Step 3 : affiche le contenu de index.html dans les logs
      # Permet de vérifier visuellement ce qui est présent sur le runner
      - name: Display index.html content
        run: cat app/index.html

# Step 4 : affiche les variables d'environnement et les secrets dans les logs
# Les secrets apparaissent censurés avec *** dans les logs GitHub
- name: Display environment variables and secrets
  env:
    FAKE_API_TOKEN: ${ secrets.FAKE_API_TOKEN } # secret GitHub
    ENV_TYPE: ${ secrets.ENV_TYPE } # secret GitHub
    APP_NAME: gtacd # variable personnalisée non-secrète
  run: |
    echo "Token API : $FAKE_API_TOKEN"
    echo "Type d'environnement : $ENV_TYPE"
    echo "Nom de l'application : $APP_NAME"
    echo "OS du runner : $RUNNER_OS"
```

```

MINGW64/c:/Users/leonc/Desktop/gtacd/gtacd
GNU nano 8.7 .github/workflows/ci_cd_full.yml Modified
# Step 2 : exécute le script de vérification
# Si le script retourne exit 1, le pipeline s'arrête ici
- name: Run application check script
  run: bash ./app/check_app.sh

# Step 3 : affiche le contenu de index.html dans les logs
# Permet de vérifier visuellement ce qui est présent sur le runner
- name: Display index.html content
  run: cat app/index.html

# Step 4 : affiche les variables d'environnement et les secrets dans les logs
# Les secrets apparaissent censurés avec *** dans les logs GitHub
- name: Display environment variables and secrets
  env:
    FAKE_API_TOKEN: ${ secrets.FAKE_API_TOKEN } # secret GitHub
    ENV_TYPE: ${ secrets.ENV_TYPE } # secret GitHub
    APP_NAME: gtacd # variable personnalisée non-secrète
  run: |
    echo "Token API : $FAKE_API_TOKEN"
    echo "Type d'environnement : $ENV_TYPE"
    echo "Nom de l'application : $APP_NAME"
    echo "OS du runner : $RUNNER_OS"

# Job de déploiement - s'exécute uniquement si build-and-test réussit
deploy:
  runs-on: ubuntu-latest
  needs: build-and-test

  steps:
    # Step 1 : clone le dépôt sur le runner
    - name: Checkout repository
      uses: actions/checkout@v4

    # Step 2 : charge la clé SSH privée dans l'agent SSH du runner
    - name: Set up SSH agent
      uses: webfactory/ssh-agent@v0.9.0
      with:
        ssh-private-key: ${ secrets.SSH_PRIVATE_KEY }

    # Step 3 : crée dynamiquement le fichier inventory.ini sur le runner
    - name: Create Ansible inventory file
      run: |
        echo "[webservers]" > inventory.ini
        echo "${ secrets.SERVER_IP } ansible_user=${ secrets.SERVER_USER }" >> inventory.ini
        chmod 600 inventory.ini

    # Step 4 : installe Ansible sur le runner
    - name: Install Ansible
      run: pip install ansible

    # Step 5 : exécute le playbook de déploiement
    - name: Run Ansible playbook
      env:
        SERVER_IP: ${ secrets.SERVER_IP }
        SERVER_USER: ${ secrets.SERVER_USER }
      run: |
        ansible-playbook -i inventory.ini ansible/deploy.yml \
          --ssh-extra-args="-o StrictHostKeyChecking=no"

AG Help      AO Write Out  AF Where Is   AK Cut        AT Execute    AC Location   M-U Undo
AX Exit      AR Read File  A\ Replace    AU Paste      AJ Justify    A/ Go To Line M-E Redo

```

```

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)
$ git add .
warning: in the working copy of 'ansible/deploy.yml', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'ansible/inventory.ini', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'ansible/nginx_vhost.conf.j2', LF will be replaced by CRLF the next time Git touches it

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)
$ git commit -m "feat: ajout du playbook Ansible et du template nginx"
[main 9d8f613] feat: ajout du playbook Ansible et du template nginx
3 files changed, 71 insertions(+)
create mode 100644 ansible/deploy.yml
create mode 100644 ansible/inventory.ini
create mode 100644 ansible/nginx_vhost.conf.j2

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)
$ git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 18 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 1.22 KiB | 624.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/leon-camuzat/gtacd.git
fffd96e..9d8f613  main -> main

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)
$

```

```

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)
$ git add .
git commit -m "ci: ajout du job de deploiement CD avec Ansible"
git push origin main
warning: in the working copy of '.github/workflows/ci_cd_full.yml', LF will be replaced by CRLF the next time Git touches it
[main ef31a98] ci: ajout du job de deploiement CD avec Ansible
1 file changed, 36 insertions(+)
rename .github/workflows/{ci_basic.yml => ci_cd_full.yml} (57%)
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 18 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 1.64 KiB | 837.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/leon-camuzat/gtacd.git
9d8f613..ef31a98  main -> main

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)
$ |

```

Documentation_GitHub_Actions

github.com/leon-camuzat/gtacd/actions

Actions

- All workflows
- Basic CI Workflow
- Secure Deployment Simulation
- Management
 - Caches
 - Attestations
 - Runners
 - Usage metrics
 - Performance metrics

Workflow runs

Workflow	Status	Branch	Run by	Time	Duration
ci: ajout du job de deployment CD avec Ansible	Failed	main	leon-camuzat	2 minutes ago	51s
ci: ajout du job de deployment CD avec Ansible	Success	main	leon-camuzat	2 minutes ago	21s
feat: ajout du playbook Ansible et du template nginx	Success	main	leon-camuzat	8 minutes ago	15s
feat: ajout du playbook Ansible et du template nginx	Success	main	leon-camuzat	8 minutes ago	10s
test: ajout du script de validation locale	Success	main	leon-camuzat	Today at 15:30	14s
test: ajout du script de validation locale	Success	main	leon-camuzat	Today at 15:30	18s
ci: ajout du workflow de simulation de deployment securise	Success	main	leon-camuzat	Today at 15:23	16s
ci: ajout du workflow de simulation de deployment securise	Success	main	leon-camuzat	Today at 15:23	11s
fix: remise en etat du script de verification	Success	main	leon-camuzat	Today at 11:57	10s
test: simulation d'un echec volontaire du pipeline	Failed	main	leon-camuzat	Today at 11:54	14s

ci: ajout du job de deployment CD avec Ansible #1

Summary

All jobs

- build-and-test
- deploy

Run details

- Usage
- Workflow file

Triggered via push 3 minutes ago

Status: Failure

Total duration: 51s

Artifacts: -

ci_cd_full.yml

on: push

build-and-test (4s) → deploy (41s)

Annotations

1 error and 2 warnings

- deploy: Process completed with exit code 4.
- build-and-test: Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4. For more information see: ...
- deploy: Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4. For more information see: ...

The screenshot shows a GitHub Actions workflow run for a job named 'deploy'. The job failed 2 minutes ago in 41s. The workflow steps are:

- Set up job (1s)
- Checkout repository (1s)
- Set up SSH agent (0s)
- Create Ansible inventory file (0s)
- Install Ansible (25s)
- Run Ansible playbook (11s) - **Failed**
- Post Set up SSH agent (0s)
- Post Checkout repository (0s)
- Complete job (0s)

The 'Run Ansible playbook' step failed with the following output:

```

1 ▶ Run ansible-playbook -i inventory.ini ansible/deploy.yml \
11
12 PLAY [Déploiement de l'application GTACD] *****
13
14 TASK [Gathering Facts] *****
15 [ERROR]: Task failed: Failed to connect to the host via ssh: ssh: connect to host *** port 22: Connection timed out
16 fatal: [***]: UNREACHABLE! => {"changed": false, "msg": "Task failed: Failed to connect to the host via ssh: ssh: connect to host *** port 22:
    Connection timed out", "unreachable": true}
17
18 PLAY RECAP *****
19 ***          : ok=0    changed=0    unreachable=1    failed=0    skipped=0    rescued=0    ignored=0
20
21 Error: Process completed with exit code 4.

```

Note : Le job `deploy` échoue en timeout SSH en raison du portail captif réseau de l'école. Les concepts et la configuration sont validés par le workflow de simulation (Job 4).

Sommaire

Test du cycle CI/CD complet

On modifie le fichier HTML pour mettre "V2.0", on pousse sur GitHub, et on regarde si le pipeline se déclenche tout seul, passe les tests, et déploie la nouvelle version sur le serveur sans qu'on touche à rien. C'est la validation finale que la chaîne complète fonctionne.

Nous modifions `index.html` pour passer en version 2 et poussons le changement sur `main`. Le pipeline se déclenche automatiquement, exécute les tests, puis déploie la nouvelle version sur le serveur. Cela valide la chaîne complète et illustre les avantages de l'automatisation : traçabilité, réduction des erreurs humaines et rollback facilité.

Mise à jour de `app/index.html` en version 2 :

```
<h1>V2.0 Déployé !</h1>
```

Le pipeline se déclenche automatiquement. Le job `deploy` ne s'exécute que si `build-and-test` passe, garantissant qu'aucune version cassée n'atteint le serveur.

```

MINGW64:/c/Users/leonc/Desktop
GNU nano 8.7 app/index.html Modified
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>GTACD - App V2.0</title>
</head>
<body>
  <h1>Bienvenue sur l'application GTACD - Version 2.0</h1>
  <p>Pipeline CI/CD securise avec GitHub Actions et Ansible.</p>
  <p>Mise a jour automatique via le pipeline - V2.0</p>
  <!-- Date de generation : sera mise a jour automatiquement par le pipeline -->
</body>
</html>

```

```

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)
$ git add app/index.html
git commit -m "feat: mise a jour de l'application en V2.0"
git push origin main
warning: in the working copy of 'app/index.html', LF will be replaced by CRLF the next time Git touches it
[main 34443df] feat: mise a jour de l'application en V2.0
1 file changed, 5 insertions(+), 4 deletions(-)
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 18 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 584 bytes | 584.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/leon-camuzat/gtacd.git
ef31a98..34443df main -> main

leonc@C56EK3PBREMR5 MINGW64 ~/Desktop/gtacd/gtacd (main)
$

```

	feat: mise a jour de l'application en V2.0	main	now 12s	...
Secure Deployment Simulation #5: Commit 34443df pushed by leon-camuzat				
	feat: mise a jour de l'application en V2.0	main	now In progress	...
Basic CI Workflow #2: Commit 34443df pushed by leon-camuzat				

deploy
failed now in 37s

Q Search logs

> Set up job

1s

> Checkout repository

0s

> Set up SSH agent

1s

> Create Ansible inventory file

0s

> Install Ansible

21s

> Run Ansible playbook

11s

```

1 ▶ Run ansible-playbook -i inventory.ini ansible/deploy.yml \
11
12 PLAY [Déploiement de l'application GTACD] *****
13
14 TASK [Gathering Facts] *****
15 [ERROR]: Task failed: Failed to connect to the host via ssh: ssh: connect to host *** port 22: Connection timed out
16 fatal: [***]: UNREACHABLE! => {"changed": false, "msg": "Task failed: Failed to connect to the host via ssh: ssh: connect to host *** port 22: Connection timed out", "unreachable": true}
17
18 PLAY RECAP *****
19 *** : ok=0 changed=0 unreachable=1 failed=0 skipped=0 rescued=0 ignored=0
20
21 Error: Process completed with exit code 4.

```

> Post Set up SSH agent

0s

> Post Checkout repository

0s

> Complete job

0s

Avantages sécurité vs déploiement manuel :

- Réduction des erreurs humaines (oubli de redémarrer nginx, mauvaises permissions...)
- Traçabilité complète : chaque déploiement est lié à un commit, un auteur, un timestamp
- Rollback facilité : on revert le commit, le pipeline redéploie l'ancienne version

[Sommaire](#)

Bilan et perspectives

Ce qui a été mis en place

- Pipeline CI/CD automatisé sur push vers `main`
- Gestion sécurisée des secrets (SSH, IP, user) via GitHub Secrets
- Utilisateur de déploiement à moindre privilège (`deployuser`)
- Clé SSH dédiée au projet, sans passphrase, révocable indépendamment
- Playbook Ansible idempotent pour le déploiement Nginx
- Workflow de simulation validant la configuration sans accès réseau externe

Pour aller plus loin

SAST/SCA : Intégrer des outils comme `semgrep` ou `trivy` dans le job CI pour détecter les vulnérabilités dans le code source et les dépendances avant tout déploiement.

DAST : Lancer un scan OWASP ZAP automatisé après déploiement en environnement de staging.

Audits et logs : Les logs GitHub Actions et Ansible constituent une trace d'audit exploitable pour la détection d'anomalies. Les intégrer dans un SIEM (Splunk, Wazuh) permet une corrélation d'incidents.

Gestion des environnements : En production : environnements GitHub avec approbation manuelle obligatoire avant le déploiement, secrets scoped par environnement, principe du moindre privilège strict.

Rollback : Stratégie de retour arrière : un tag git + un workflow `rollback.yml` déclenché manuellement qui redéploie le dernier tag stable.

[Sommaire](#)

#01